



Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA

O Choque de Realidades (Conflitos)

- Imagine que na linha do tempo principal (a main), a parede da sala é Branca. No Universo A, você pintou a parede da sala de **Azul**. No Universo B, seu colega pintou a MESMA parede de **Vermelho**.
 - Quando tentamos juntar os universos no GitHub, o que acontece?
 - A parede fica roxa?
 - Metade azul, metade vermelha?
 - Não. O Git para o universo e chama o Juiz (Você).
- **O Conflito:**
 - O Git para e diz: *"Sou uma máquina, não sei de qual cor você gosta mais. Escolha você!"*.
 - O VS Code vai ficar colorido mostrando as duas opções para o programador decidir.

O Choque de Realidades (Conflitos)

- Não. O Git para o universo e chama o Juiz (Você).
- **O Conflito:**
 - O Git para e diz: *"Sou uma máquina, não sei de qual cor você gosta mais. Escolha você!"*.
 - O VS Code vai ficar colorido mostrando as duas opções para o programador decidir.

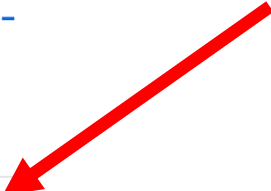
Novo Projeto

<https://classroom.github.com/a/m8oP4pjd>

Turma-GCV-2026-1

Accept the assignment — Tarefa - Conflito

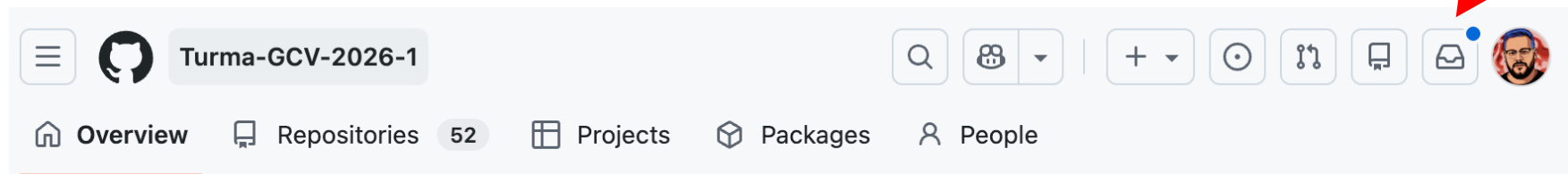
Once you accept this assignment, you will be granted access to the `tarefa-conflito-cassiocapucho` repository in the [Turma-GCV-2026-1](#) organization on GitHub.



Accept this assignment

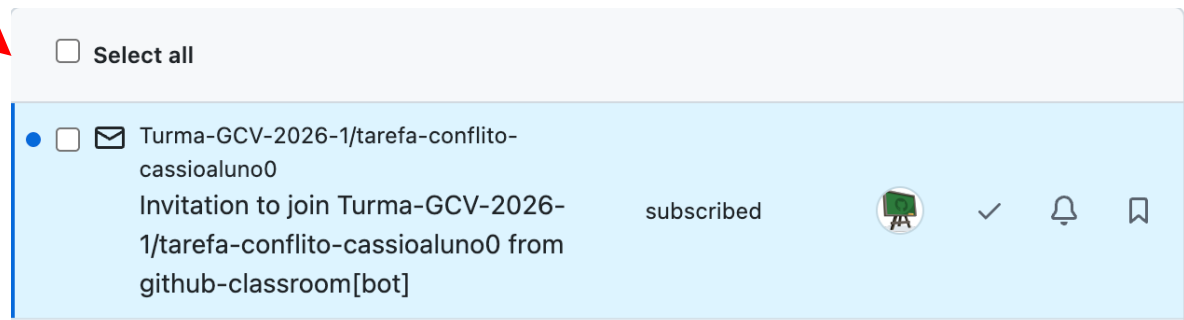
Novo Projeto

Você receberá uma notificação no seu github.



Turma GCV 2026.1

Follow



Novo Projeto

Aceite o convite...



[github-classroom\[bot\]](#) invited you to collaborate on
Turma-GCV-2026-1/tarefa-conflito-cassioaluno0



Accept invitation

Decline invitation

🔒 [Owners](#) of tarefa-conflito-cassioaluno0 will be able to see:

- Your public profile information
- [Certain activity](#) within this repository
- Country of request origin
- Your access level for this repository
- Your IP address

Is this user sending spam or malicious content?

[Block github-classroom](#)

Novo Projeto

Crie um novo Codespace.

The screenshot shows a GitHub repository named 'tarefa-conflito-cassioaluno0' (Public) forked from 'Turma-GCV-2026-1/turma-gcv-2026-1-tarefa-conflito-repositorio_base'. The repository is on the 'main' branch. The 'Codespaces' tab is selected, showing a message: 'No codespaces. You don't have any codespaces with this repository checked out'. A red arrow points to the 'Create codespace on main' button. Below this, there is a link 'Learn more about codespaces...'. At the bottom, it states 'Codespace usage for this repository is paid for by cassioaluno0.'.

tarefa-conflito-cassioaluno0 Public

forked from [Turma-GCV-2026-1/turma-gcv-2026-1-tarefa-conflito-repositorio_base](#)

main

Go to file + <> Code

This branch is 1 commit ahead of [Turma-GCV-2026-1/turma-gcv-2026-1-tarefa-conflito-repositorio_base](#)

Contribute Sync fork

github-classroom[bot] add online IDE

.gitignore

README.md

main.c

README

Open in Visual Studio Code

Codespaces

Your workspaces in the cloud

No codespaces

You don't have any codespaces with this repository checked out

Create codespace on main

[Learn more about codespaces...](#)

Codespace usage for this repository is paid for by cassioaluno0.

Próximos passos...

- Existe o projeto original (professor), o seu fork (seu GitHub) e a sua máquina (Codespace).
- Eu vou atualizar o projeto original.
- Você recebe a notificação no seu GitHub.
- Aceite receber as atualizações no seu GitHub.
- Seu Codespace ainda não estará atualizado.
- Execute: **git status**
 - Não estava atualizado?

O Radar da Nuvem - **git fetch**

- Como descobrir as novidades sem estragar o seu código?
- **O que faz:**
 - Atualiza a "fotografia" que o seu computador tem do GitHub.
 - Ele baixa as fofocas (o histórico novo), mas **NÃO** altera seus arquivos e **NÃO** gera conflitos.
 - É o comando que cura a "Mentira do git status", forçando o Git a olhar para a internet.
- **A Matemática do Git:** `git pull = git fetch + git merge`
- Execute 01: **git fetch**
- Execute02: **git status**

O tradicional "Merge"

➤ `git config pull.rebase false`

- Esta é a opção clássica e historicamente o padrão do Git.
- Ele pega o seu trabalho local e o trabalho que vem do GitHub e cria um **novo commit** unindo os dois (o famoso "Merge Commit").
- Imagine uma estrada que se divide em duas e depois se junta novamente mais à frente.
 - É a ideal para iniciantes entenderem que duas coisas paralelas foram fundidas.
- O lado negativo é que, em equipes grandes, o histórico do repositório fica cheio de commits de merge, parecendo uma "teia de aranha".

A reescrita "Rebase"

➤ `git config pull.rebase true`

- Esta é a opção preferida por equipes mais experientes que gostam de um histórico limpo.
- Ele "pausa" os commits que você fez na sua máquina, baixa as novidades do GitHub, e depois **reaplica** os seus commits no topo da fila, um por um.
 - Fica uma linha perfeitamente reta. Ele esconde que houve um trabalho em paralelo.
- É uma abordagem mais avançada e pode gerar conflitos sequenciais mais difíceis de resolver para quem está começando, pois ele reescreve a história do Git.

O modo de segurança "Fast-forward"

➤ `git config pull.ff only`

- Esta é a opção mais restrita.
- Ele só aceita fazer o pull se a sua linha do tempo local estiver exatamente atrás da remota.
 - ou seja, se ninguém mexeu em paralelo.
- Como as linhas divergiram, se você escolher essa opção, o Git simplesmente **aborta a operação** e não faz nada, obrigando você a resolver manualmente.
 - digitando `git merge` ou `git rebase` na mão.
- É ótimo para evitar acidentes, mas vai muito burocrático.

Comando **--global**

- O Git trabalha com níveis de configuração.
 - Essa “dica” no final explica onde a sua escolha será salva
- **Se você usar apenas git config ... (sem o global):**
 - A regra que você escolher valerá **apenas para esta pasta**.
 - Se você criar outro projeto amanhã, o Git vai fazer a mesma pergunta de novo.
- **Se você usar git config --global ...:**
 - A regra será salva no seu usuário do computador/Codespace.
 - A partir de agora, **todos os repositórios** que você usar nessa máquina seguirão essa regra silenciosamente, sem nunca mais te perguntar.



git



Práticas

Cenário Prático 1: O Caminho Feliz (A Mágica do Sumiço)

Verifique o terreno: Rode `git branch` (Vai listar apenas a main com um asterisco verde, indicando onde você está).

Crie o Universo Paralelo: `git switch -c nova-tela`

Trabalhe seguro: Crie um arquivo chamado `tela_login.html` e digite qualquer coisa.

Salve a foto: Rode `git add .` e `git commit -m "Criei a tela de login"`.

O Momento Mágico: "Pessoal, olhem para a barra lateral do VS Code onde ficam os arquivos. Agora digitem `git switch main` no terminal". ✨ (A `tela_login.html` vai desaparecer instantaneamente da tela deles! É aqui que eles entendem o isolamento).

O Resgate (Merge): "Calma, não apagou! Voltemos para o terminal e vamos puxar o trabalho: digitem `git merge nova-tela`". ✨ (O arquivo ressurge magicamente na pasta deles, agora integrado à main).

Cenário Prático 2:

Provocando um Conflito de Propósito

Para perderem o medo do erro vermelho de *Conflict*.

Prepare a Base: No arquivo main.c (na branch main), escreva na linha 1: `printf("Ola Mundo");`, dê add e commit.

O Ramo A: Digite `git switch -c ramo-a`. Mude a linha 1 para `printf("Ola Galera");`. Dê add e commit.

O Ramo B: Volte para a base (`git switch main`). Mude a MESMA linha 1 para `printf("Ola Turma");`. Dê add e commit.

A Colisão: Ainda na main, tente puxar o ramo A: `git merge ramo-a`.

O Diagnóstico: O terminal vai gritar: `CONFLICT (content): Merge conflict in main.c`.

A Resolução: Peça para eles olharem o VS Code. O arquivo estará cheio de setas <<<<<< e botões de atalho verde e azul (Accept Current Change / Accept Incoming Change). Ensine-os a clicar no botão para escolher a versão final, salvar o arquivo, e finalizar o processo com `git add .` e `git commit -m "Conflito resolvido"`.